

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: Design and Analysis of Algorithms
CSA06

COURSE CODE:

COURSE OUTCOME COVERED

CO2: Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1,BL2,BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 10

Total Marks: 50

Annexure A

Scenario Based Assessment

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario-Based Assessment

1. A university maintains student records where marks are almost sorted, except for a few recently updated entries. The system needs to sort the data frequently with minimal computation time. The developer is choosing between Selection Sort, Bubble Sort, and Insertion Sort.

- a) Analyze all three algorithms in this context.
- b) Recommend the most suitable one with justification based on comparisons and swaps.

2. A logistics company manages 500 warehouse locations. To optimize delivery routes, the system calculates the distance between every pair of warehouses to identify the closest pair. As the number of warehouses increases, the system becomes slower.

- a) Explain how the brute-force approach works in this case.
- b) Derive the time complexity.
- c) Discuss why scalability is an issue.

3. A hospital database stores patient records in an unsorted list because records are frequently added and updated. Doctors need to quickly search for patient details during emergencies.

- a) Explain how sequential search works in this scenario.
- b) Analyze best, worst, and average case performance.
- c) Justify why sorting is not preferred here.

4. A GIS application determines the boundary of a region by identifying outermost points using a brute-force convex hull approach.

- a) Explain how the brute-force convex hull algorithm works.
- b) Analyze its computational complexity.
- c) Discuss why it becomes inefficient for large datasets.

5. A plagiarism detection system compares a pattern against a large document using brute-force string matching. The document length is very large, and the pattern appears rarely.

- a) Explain the working of brute-force string matching.

- b) Analyze its time complexity.
- c) Discuss why performance degrades in this scenario.

6. A retail company maintains daily sales data that must be searched quickly to find specific transaction IDs. The dataset is unsorted due to continuous updates. The system currently uses Linear Search, but performance degrades as data grows.

- a) Explain how Linear Search operates in this scenario.
- b) Analyze the time complexity for best, average, and worst cases.
- c) Suggest whether sorting + Binary Search would improve performance and justify your decision.

7. A navigation system processes coordinates of multiple delivery points to identify the farthest pair of locations for route planning. The system currently checks all possible pairs using a brute-force approach.

- a) Explain how the brute-force method works for this problem.
- b) Derive the time complexity for computing all pair distances.
- c) Analyze why this approach becomes inefficient as the number of points increases.

8. A text editor searches for a keyword within large documents using brute-force string matching. As document size increases, search time becomes significantly high.

- a) Explain how brute-force string matching works in this context.
- b) Analyze the time complexity of the algorithm.
- c) Suggest why this approach is inefficient and justify the need for optimized algorithms.

9. A dataset of student marks is stored in sorted order, and the system frequently needs to locate a specific mark using Binary Search. However, new records are inserted randomly, disturbing the sorted order.

- a) Explain how Binary Search works when the data is sorted.
- b) Analyze its time complexity in best, average, and worst cases.
- c) Discuss why maintaining sorted order is critical and justify the impact on search performance.

10. A robotics system analyzes multiple obstacle points in a 2D plane to determine the boundary region using a brute-force approach. As the number of points increases, processing time becomes high.

- a) Explain how brute-force geometric analysis (similar to convex hull) works.
- b) Analyze the time complexity of checking all point combinations.
- c) Discuss why this approach is not scalable and justify the need for efficient alternatives.

Q11. A banking system stores daily transaction amounts in a list where new entries are appended randomly, causing disorder. The system needs to sort data periodically for reporting.

- a) Analyze how Selection Sort and Bubble Sort would work in this scenario.
- b) Compare the number of comparisons and swaps required by both methods.
- c) Recommend the most suitable sorting technique with proper justification.

Q12. A delivery company maintains GPS coordinates of delivery points and computes distances between every pair to find the closest drop location. As the number of locations increases, delays are observed.

- a) Explain how the brute-force closest pair algorithm operates in this scenario.
- b) Derive the time complexity of computing all pair distances.
- c) Analyze why the performance degrades with large datasets.

Q13. A university portal stores student IDs in an unsorted format due to continuous registrations. During exams, quick lookup of student IDs is required.

- a) Explain how Linear Search works for retrieving student records.
- b) Analyze best, average, and worst-case time complexities.
- c) Justify whether sorting the dataset would improve performance.

Q14. A mapping application determines the outer boundary of a set of plotted land coordinates using a brute-force convex hull method. The dataset size increases significantly over time.

- a) Explain how the brute-force convex hull algorithm works in this case.
- b) Analyze the computational complexity of checking all point pairs.
- c) Discuss why this approach becomes inefficient for large inputs.

Q15. A plagiarism detection system scans large documents to match a given pattern using brute-force string matching. The document size is very large and pattern occurrences are rare.

- a) Explain how brute-force string matching operates in this scenario.
- b) Analyze its time complexity.
- c) Discuss why the approach becomes inefficient for large text inputs.

Q16. A retail database stores product IDs in unsorted order due to frequent updates. The system uses Linear Search to find products, but response time increases with data size.

- a) Explain how Linear Search works in this context.
- b) Analyze the time complexity in all cases.
- c) Suggest whether switching to Binary Search would improve performance and justify.

Q17. A navigation system evaluates multiple city coordinates to identify the farthest pair using brute-force comparison of all pairs.

- a) Explain how the brute-force approach is applied in this problem.
- b) Derive the time complexity for evaluating all pairs.
- c) Analyze why this approach is inefficient for large datasets.

Q18. A text editor searches for keywords in large files using brute-force string matching. Users experience delays as file sizes grow.

- a) Explain how the brute-force string matching algorithm works.
- b) Analyze the time complexity of this approach.
- c) Justify why optimized algorithms may be required.

Q19. A database maintains sorted employee salary records, allowing Binary Search for quick retrieval. However, frequent updates disturb the sorted order.

- a) Explain how Binary Search works when data is sorted.
- b) Analyze its time complexity.
- c) Discuss the importance of maintaining sorted data for efficient searching.

Q20. A robotics system detects obstacles by analyzing coordinate points and determining the boundary using brute-force geometric techniques.

- a) Explain how brute-force boundary detection (convex hull concept) works.
- b) Analyze the time complexity of checking all point combinations.
- c) Discuss why this approach is not scalable.

Q21. A financial system processes stock prices and needs to sort them frequently for trend analysis, but the data changes continuously.

- a) Compare Selection Sort and Bubble Sort for this scenario.
- b) Analyze their computational cost in terms of comparisons.
- c) Recommend a suitable method with justification.

Q22. A transportation system calculates distances between bus stops to identify the closest stops for route optimization. The number of stops increases daily.

- a) Explain how brute-force closest pair works here.
- b) Derive its time complexity.
- c) Discuss scalability issues.

Q23. A hospital system stores emergency patient data unsorted for quick updates. Doctors need to locate records rapidly.

- a) Explain how Linear Search helps in this scenario.
- b) Analyze its performance in different cases.
- c) Justify why sorting may not always be preferred.

Q24. A satellite imaging system processes geographic points to determine outer boundaries using convex hull techniques.

- a) Explain how brute-force convex hull identifies boundary points.
- b) Analyze the computational complexity.

c) Discuss limitations of this approach.

Q25. A document scanning tool matches keywords using brute-force string matching in large files.

- a) Explain how pattern matching is performed.
- b) Analyze time complexity for large inputs.
- c) Discuss inefficiency issues.

Q26. An e-commerce system stores order IDs unsorted due to continuous transactions and uses Linear Search for retrieval.

- a) Explain how Linear Search operates in this case.
- b) Analyze performance degradation as data grows.
- c) Suggest improvements and justify.

Q27. A weather monitoring system compares coordinates of stations to find the nearest ones using brute-force.

- a) Explain how all pairs are evaluated.
- b) Derive time complexity.
- c) Discuss inefficiency for large-scale systems.

Q28. A software tool searches for a substring in logs using brute-force string matching. Logs are very large.

- a) Explain how the matching process works.
- b) Analyze its time complexity.
- c) Justify need for optimization.

Q29. A sorted dataset is used in a banking system for quick balance lookup using Binary Search, but frequent updates disturb order.

- a) Explain Binary Search operation.
- b) Analyze performance when order is maintained vs disturbed.
- c) Discuss importance of sorted structure.

Q30. A drone navigation system determines boundary points of obstacles using brute-force convex hull approach.

- a) Explain how convex hull is computed.
- b) Analyze time complexity.
- c) Discuss scalability challenges.

Q31. A data analytics platform processes sensor readings that arrive in random order and must be sorted periodically for trend visualization.

- a) Analyze how Selection Sort and Bubble Sort would behave for this dataset.
- b) Compare their performance in terms of comparisons and swaps.
- c) Recommend the most suitable sorting method with proper justification.

Q32. A ride-sharing application calculates distances between all driver and rider locations to assign the nearest driver using brute-force comparison.

- a) Explain how the brute-force closest pair approach works in this case.
- b) Derive the time complexity for computing all distances.
- c) Discuss why scalability becomes an issue as users increase.

Q33. A library system stores book IDs in an unsorted list due to frequent additions and deletions. Users search for books using a basic search feature.

- a) Explain how Linear Search is applied in this scenario.
- b) Analyze the best, average, and worst-case complexities.
- c) Justify whether sorting would improve system efficiency.

Q34. A geographic mapping system identifies the outer boundary of terrain points using a brute-force convex hull algorithm. The number of data points grows continuously.

- a) Explain how the brute-force convex hull algorithm operates.
- b) Analyze its computational complexity.
- c) Discuss why the approach is inefficient for large datasets.

Q35. A cybersecurity system scans large log files to detect specific attack patterns using brute-force string matching.

- a) Explain how brute-force string matching works in this context.
- b) Analyze the time complexity of the algorithm.
- c) Discuss why performance issues arise with large logs.

Q36. A retail chain maintains a product list that is frequently updated and unsorted. The system searches for products using Linear Search.

- a) Explain how Linear Search operates in this system.
- b) Analyze its performance as dataset size increases.
- c) Suggest whether Binary Search would be beneficial and justify.

Q37. A satellite tracking system evaluates coordinates of objects in space to find the closest pair using brute-force comparison.

- a) Explain how all coordinate pairs are evaluated.
- b) Derive the time complexity of the brute-force method.
- c) Discuss the limitations when the number of objects increases.

Q38. A document editor uses brute-force pattern matching to locate keywords in large research papers.

- a) Explain how the pattern matching process works.
- b) Analyze the time complexity of this approach.
- c) Justify the need for more efficient algorithms.

Q39. A financial system maintains sorted transaction data for quick lookup using Binary Search, but frequent updates disrupt the order.

- a) Explain how Binary Search operates on sorted data.
- b) Analyze its performance when order is maintained versus disturbed.

- c) Discuss the importance of maintaining sorted data.

Q40. A robotics system processes obstacle points to determine the outer boundary using brute-force geometric analysis.

- a) Explain how convex hull concepts apply here.
- b) Analyze the time complexity of evaluating all point combinations.
- c) Discuss scalability concerns.

Q41. A logistics company needs to sort delivery times stored in an unsorted list before generating reports.

- a) Compare Selection Sort and Bubble Sort for this scenario.
- b) Analyze the number of comparisons required.
- c) Recommend the better algorithm with justification.

Q42. A city planning system computes distances between all public service centers to identify the closest ones.

- a) Explain how brute-force closest pair is applied.
- b) Derive the time complexity.
- c) Discuss performance limitations.

Q43. A student database stores records unsorted due to continuous updates. The system retrieves student details using sequential search.

- a) Explain how Linear Search works here.
- b) Analyze its efficiency in different cases.
- c) Justify whether sorting would help.

Q44. A land survey application determines the boundary of a region using convex hull algorithms.

- a) Explain how brute-force convex hull identifies boundary points.
- b) Analyze its time complexity.
- c) Discuss why it is inefficient for large datasets.

Q45. A plagiarism detection tool compares patterns across multiple large documents using brute-force string matching.

- a) Explain how the matching process works.
- b) Analyze the time complexity.
- c) Discuss inefficiency in large-scale comparisons.

Q46. An inventory system searches for item IDs in an unsorted dataset using Linear Search.

- a) Explain how Linear Search is applied.
- b) Analyze performance degradation with large data.
- c) Suggest improvements and justify.

Q47. A communication network calculates distances between nodes to identify the closest pair using brute-force computation.

- a) Explain how all node pairs are evaluated.
- b) Derive the time complexity.
- c) Discuss scalability issues.

Q48. A search engine scans large text data to find keywords using brute-force pattern matching.

- a) Explain the working of brute-force string matching.
- b) Analyze its time complexity.
- c) Justify the need for optimized search algorithms.

Q49. A banking system uses Binary Search on sorted customer IDs, but frequent insertions disturb order.

- a) Explain how Binary Search works.
- b) Analyze the impact of unsorted data.
- c) Discuss the importance of maintaining sorted datasets.

Q50. A drone system identifies boundary points of obstacles using brute-force convex hull computation.

- a) Explain how convex hull is determined.
- b) Analyze the time complexity.
- c) Discuss why this approach is not scalable.

Q51. A smart grid system collects power consumption readings that arrive in random order and must be sorted periodically for analysis.

- a) Analyze how Selection Sort and Bubble Sort perform on this dataset.
- b) Compare their efficiency in terms of comparisons and swaps.
- c) Recommend the most suitable algorithm with justification.

Q52. A food delivery platform computes distances between restaurants and customers to assign the nearest delivery partner using brute-force comparison.

- a) Explain how the brute-force closest pair approach works in this context.
- b) Derive the time complexity for computing all distances.
- c) Discuss scalability issues as the number of users increases.

Q53. A university system stores attendance records unsorted due to frequent updates. Faculty need to quickly locate specific student records.

- a) Explain how Linear Search works in this scenario.
- b) Analyze its performance in best, average, and worst cases.
- c) Justify whether sorting would improve efficiency.

Q54. A GIS-based flood monitoring system determines the outer boundary of flooded regions using convex hull techniques.

- a) Explain how brute-force convex hull is applied.
- b) Analyze its computational complexity.

c) Discuss limitations when handling large datasets.

Q55. A digital library searches for keywords in large research articles using brute-force string matching.

- a) Explain how pattern matching is performed.
- b) Analyze the time complexity.
- c) Discuss inefficiency in large-scale document processing.

Q56. An online shopping platform maintains product data unsorted due to dynamic updates and uses Linear Search for retrieval.

- a) Explain how Linear Search operates in this system.
- b) Analyze performance degradation with increasing data size.
- c) Suggest improvements and justify.

Q57. A transportation system calculates distances between multiple bus stops to identify the nearest stops using brute-force.

- a) Explain how all possible pairs are evaluated.
- b) Derive the time complexity.
- c) Discuss why the method is inefficient for large inputs.

Q58. A code editor searches for variables and keywords in large programs using brute-force string matching.

- a) Explain how the algorithm works.
- b) Analyze its time complexity.
- c) Justify the need for optimized string searching methods.

Q59. A payroll system maintains sorted employee IDs for quick lookup using Binary Search, but frequent additions disturb the order.

- a) Explain how Binary Search works.
- b) Analyze the impact of unsorted data on performance.
- c) Discuss the importance of maintaining sorted datasets.

Q60. A drone surveillance system detects the outer boundary of objects using brute-force convex hull computation.

- a) Explain how convex hull is determined.
- b) Analyze the time complexity.
- c) Discuss scalability challenges.

Q61. A weather forecasting system processes temperature readings that must be sorted before analysis. Data arrives in random order.

- a) Compare Selection Sort and Bubble Sort for this dataset.
- b) Analyze their computational cost.
- c) Recommend the better algorithm with justification.

Q62. A telecom network evaluates distances between towers to find the closest pair for signal optimization.

- a) Explain how brute-force closest pair works.
- b) Derive the time complexity.
- c) Discuss performance limitations.

Q63. A hospital database stores emergency records unsorted due to continuous updates. Doctors search patient details during emergencies.

- a) Explain how Linear Search works.
- b) Analyze its efficiency.
- c) Justify whether sorting is beneficial.

Q64. A mapping system determines the outermost boundary of terrain using convex hull algorithms.

- a) Explain how brute-force convex hull identifies boundary points.
- b) Analyze the time complexity.
- c) Discuss inefficiency for large datasets.

Q65. A plagiarism detection engine scans documents using brute-force pattern matching to find similarities.

- a) Explain how the algorithm works.
- b) Analyze its time complexity.
- c) Discuss inefficiency for large documents.

Q66. A warehouse management system searches for product IDs in an unsorted dataset using Linear Search.

- a) Explain how Linear Search is applied.
- b) Analyze performance issues.
- c) Suggest improvements and justify.

Q67. A navigation system evaluates distances between delivery points to find the closest ones using brute-force.

- a) Explain how all pairs are evaluated.
- b) Derive the time complexity.
- c) Discuss scalability concerns.

Q68. A search tool scans large log files to find error patterns using brute-force string matching.

- a) Explain the working of the algorithm.
- b) Analyze the time complexity.
- c) Justify the need for optimization.

Q69. A financial system uses Binary Search for sorted transaction records, but frequent updates disturb order.

- a) Explain Binary Search operation.
- b) Analyze the impact of unsorted data.
- c) Discuss maintaining sorted data for efficiency.

Q70. A robotics application identifies the boundary of objects using brute-force convex hull computation.

- a) Explain how convex hull is computed.
- b) Analyze the time complexity.
- c) Discuss why the approach is not scalable.

Q71. A smart agriculture system collects soil moisture readings from sensors placed randomly and needs periodic sorting for irrigation planning.

- a) Analyze how Selection Sort and Bubble Sort would perform on this dataset.
- b) Compare their efficiency in terms of comparisons and swaps.
- c) Recommend the most suitable sorting technique with justification.

Q72. A ride-hailing service calculates distances between drivers and passengers using brute-force comparison to assign the nearest driver.

- a) Explain how the brute-force closest pair approach works.
- b) Derive the time complexity.
- c) Discuss scalability issues as the number of users increases.

Q73. A digital library stores book records unsorted due to frequent updates and uses sequential search to retrieve information.

- a) Explain how Linear Search works in this scenario.
- b) Analyze best, average, and worst-case performance.
- c) Justify whether sorting would improve efficiency.

Q74. A disaster management system identifies the outer boundary of affected regions using convex hull algorithms on a large set of coordinates.

- a) Explain how brute-force convex hull operates.
- b) Analyze its computational complexity.
- c) Discuss limitations when handling large datasets.

Q75. A malware detection tool scans system logs using brute-force string matching to find suspicious patterns.

- a) Explain how the pattern matching algorithm works.
- b) Analyze its time complexity.
- c) Discuss inefficiency for large log files.

Q76. An online retail system searches for customer orders stored in an unsorted list using Linear Search.

- a) Explain how Linear Search is applied.
- b) Analyze performance degradation as data grows.
- c) Suggest improvements and justify.

Q77. A GPS navigation system computes distances between all route points to determine the closest pair using brute-force.

- a) Explain how all possible pairs are evaluated.

- b) Derive the time complexity.
- c) Discuss scalability challenges.

Q78. A programming IDE searches for function names within large codebases using brute-force string matching.

- a) Explain how the matching process works.
- b) Analyze its time complexity.
- c) Justify the need for optimized searching algorithms.

Q79. A banking system maintains sorted account numbers for quick retrieval using Binary Search, but frequent updates disrupt order.

- a) Explain how Binary Search works.
- b) Analyze the effect of unsorted data on performance.
- c) Discuss why maintaining sorted order is important.

Q80. A robotics system processes coordinate points to detect object boundaries using convex hull algorithms.

- a) Explain how brute-force convex hull is applied.
- b) Analyze the time complexity.
- c) Discuss why the approach is inefficient for large inputs.

Q81. A stock market analysis system sorts fluctuating stock prices before generating reports. The data arrives in random order.

- a) Compare Selection Sort and Bubble Sort for this dataset.
- b) Analyze their computational cost.
- c) Recommend the better algorithm with justification.

Q82. A telecom system evaluates distances between towers to optimize signal strength using brute-force closest pair computation.

- a) Explain how brute-force closest pair works.
- b) Derive the time complexity.
- c) Discuss performance limitations.

Q83. A hospital system stores patient records unsorted due to continuous updates and uses Linear Search during emergencies.

- a) Explain how Linear Search operates.
- b) Analyze its efficiency in different cases.
- c) Justify whether sorting is beneficial.

Q84. A land mapping application determines outer boundaries using convex hull algorithms for terrain analysis.

- a) Explain how brute-force convex hull identifies boundary points.
- b) Analyze the time complexity.
- c) Discuss inefficiency for large datasets.

Q85. A document analysis system detects keywords using brute-force string matching in large text files.

- a) Explain how the algorithm works.
- b) Analyze the time complexity.
- c) Discuss inefficiency for large inputs.

Q86. An inventory system searches for items stored in an unsorted list using Linear Search.

- a) Explain how Linear Search is applied.
- b) Analyze performance issues.
- c) Suggest improvements and justify.

Q87. A delivery system calculates distances between drop points to identify the nearest locations using brute-force.

- a) Explain how all pairs are evaluated.
- b) Derive the time complexity.
- c) Discuss scalability concerns.

Q88. A log monitoring system scans large files to detect error patterns using brute-force string matching.

- a) Explain the working of the algorithm.
- b) Analyze its time complexity.
- c) Justify the need for optimized approaches.

Q89. A payroll system uses Binary Search on sorted employee IDs, but frequent insertions disturb the order.

- a) Explain Binary Search operation.
- b) Analyze the impact of unsorted data.
- c) Discuss maintaining sorted data for efficiency.

Q90. A drone navigation system determines object boundaries using brute-force convex hull computation.

- a) Explain how convex hull is computed.
- b) Analyze the time complexity.
- c) Discuss why the approach is not scalable.

Q91. A smart traffic control system collects vehicle counts from multiple junctions in random order and needs sorting before congestion analysis.

- a) Analyze how Selection Sort and Bubble Sort would perform on this dataset.
- b) Compare their efficiency in terms of comparisons and swaps.
- c) Recommend the most suitable sorting technique with justification.

Q92. A drone delivery system computes distances between warehouses and delivery points using brute-force to find the closest location.

- a) Explain how the brute-force closest pair algorithm works in this scenario.

- b) Derive the time complexity.
- c) Discuss scalability issues as the number of locations increases.

Q93. A university admission system stores applicant data unsorted due to continuous registrations and uses sequential search for retrieval.

- a) Explain how Linear Search works in this context.
- b) Analyze best, average, and worst-case performance.
- c) Justify whether sorting would improve efficiency.

Q94. A disaster response system maps affected zones using coordinate points and determines the boundary using convex hull techniques.

- a) Explain how brute-force convex hull operates.
- b) Analyze its computational complexity.
- c) Discuss limitations when handling large datasets.

Q95. A cybersecurity monitoring system scans large network logs using brute-force string matching to detect intrusion patterns.

- a) Explain how the pattern matching algorithm works.
- b) Analyze its time complexity.
- c) Discuss inefficiency for large-scale logs.

Q96. An e-commerce system searches for product records stored in an unsorted list using Linear Search.

- a) Explain how Linear Search operates in this system.
- b) Analyze performance degradation with increasing data size.
- c) Suggest improvements and justify.

Q97. A GPS tracking system evaluates distances between multiple vehicles to identify the closest pair using brute-force computation.

- a) Explain how all possible pairs are evaluated.
- b) Derive the time complexity.
- c) Discuss scalability challenges.

Q98. A programming tool searches for specific syntax patterns in large code repositories using brute-force string matching.

- a) Explain how the matching process works.
- b) Analyze its time complexity.
- c) Justify the need for optimized algorithms.

Q99. A banking system maintains sorted customer IDs for quick lookup using Binary Search, but frequent updates disrupt the order.

- a) Explain how Binary Search works.
- b) Analyze the effect of unsorted data on performance.
- c) Discuss the importance of maintaining sorted order.

Q100. A robotics system processes obstacle coordinates to determine boundaries using brute-force convex hull computation.

- a) Explain how convex hull is determined.
- b) Analyze the time complexity.
- c) Discuss scalability concerns.

Q101. A weather analysis system collects temperature readings in random order and needs sorting before generating forecasts.

- a) Compare Selection Sort and Bubble Sort for this dataset.
- b) Analyze their computational cost.
- c) Recommend the better algorithm with justification.

Q102. A telecom company calculates distances between signal towers to optimize connectivity using brute-force closest pair analysis.

- a) Explain how the brute-force method works.
- b) Derive the time complexity.
- c) Discuss performance limitations.

Q103. A hospital management system stores patient data unsorted and retrieves records using Linear Search during emergencies.

- a) Explain how Linear Search operates.
- b) Analyze its efficiency.
- c) Justify whether sorting is beneficial.

Q104. A land surveying application determines region boundaries using convex hull algorithms on a large dataset of coordinates.

- a) Explain how brute-force convex hull identifies boundary points.
- b) Analyze the time complexity.
- c) Discuss inefficiency for large datasets.

Q105. A document processing system scans large files using brute-force string matching to detect keywords.

- a) Explain how the algorithm works.
- b) Analyze the time complexity.
- c) Discuss inefficiency for large inputs.

Q106. An inventory management system searches for product IDs in an unsorted dataset using Linear Search.

- a) Explain how Linear Search is applied.
- b) Analyze performance issues.
- c) Suggest improvements and justify.

Q107. A logistics company computes distances between delivery hubs to find the closest ones using brute-force computation.

- a) Explain how all pairs are evaluated.

- b) Derive the time complexity.
- c) Discuss scalability concerns.

Q108. A system log analyzer scans large logs to detect patterns using brute-force string matching.

- a) Explain the working of the algorithm.
- b) Analyze its time complexity.
- c) Justify the need for optimized approaches.

Q109. A payroll system uses Binary Search on sorted employee records, but frequent insertions disturb order.

- a) Explain Binary Search operation.
- b) Analyze the impact of unsorted data.
- c) Discuss maintaining sorted data for efficiency.

Q110. A drone mapping system identifies terrain boundaries using brute-force convex hull computation.

- a) Explain how convex hull is computed.
- b) Analyze the time complexity.
- c) Discuss why the approach is not scalable.

Q111. A smart irrigation system collects water level data from multiple sensors in random order and needs sorting before analysis.

- a) Analyze how Selection Sort and Bubble Sort would perform on this dataset.
- b) Compare their efficiency in terms of comparisons and swaps.
- c) Recommend the most suitable sorting technique with justification.

Q112. A courier service calculates distances between pickup and delivery points using brute-force to assign the nearest courier.

- a) Explain how the brute-force closest pair algorithm works.
- b) Derive the time complexity.
- c) Discuss scalability issues as the number of requests increases.

Q113. A university system maintains student records in an unsorted list due to frequent updates and uses Linear Search for retrieval.

- a) Explain how Linear Search works in this scenario.
- b) Analyze best, average, and worst-case performance.
- c) Justify whether sorting would improve efficiency.

Q114. A flood monitoring system maps affected regions using coordinate points and determines the boundary using convex hull techniques.

- a) Explain how brute-force convex hull operates.
- b) Analyze its computational complexity.
- c) Discuss limitations when handling large datasets.

Q115. A text analysis tool scans large documents using brute-force string matching to detect specific keywords.

- a) Explain how the pattern matching algorithm works.
- b) Analyze its time complexity.
- c) Discuss inefficiency for large inputs.

Q116. An online marketplace searches for product details stored in an unsorted dataset using Linear Search.

- a) Explain how Linear Search operates in this system.
- b) Analyze performance degradation with increasing data size.
- c) Suggest improvements and justify.

Q117. A GPS tracking system evaluates distances between delivery vehicles to identify the closest pair using brute-force computation.

- a) Explain how all possible pairs are evaluated.
- b) Derive the time complexity.
- c) Discuss scalability challenges.

Q118. A programming tool searches for code patterns in large repositories using brute-force string matching.

- a) Explain how the matching process works.
- b) Analyze its time complexity.
- c) Justify the need for optimized algorithms.

Q119. A banking system maintains sorted transaction records for quick lookup using Binary Search, but frequent updates disrupt the order.

- a) Explain how Binary Search works.
- b) Analyze the effect of unsorted data on performance.
- c) Discuss the importance of maintaining sorted order.

Q120. A robotics system processes coordinate points to determine boundaries using brute-force convex hull computation.

- a) Explain how convex hull is determined.
- b) Analyze the time complexity.
- c) Discuss scalability concerns.

Q121. A stock analysis system collects fluctuating stock prices in random order and needs sorting before prediction analysis.

- a) Compare Selection Sort and Bubble Sort for this dataset.
- b) Analyze their computational cost.
- c) Recommend the better algorithm with justification.

Q122. A telecom network evaluates distances between towers to optimize connectivity using brute-force closest pair computation.

- a) Explain how the brute-force method works.

- b) Derive the time complexity.
- c) Discuss performance limitations.

Q123. A hospital management system stores patient records unsorted and retrieves them using Linear Search during emergencies.

- a) Explain how Linear Search operates.
- b) Analyze its efficiency.
- c) Justify whether sorting is beneficial.

Q124. A land mapping application determines the outer boundary of terrain using convex hull algorithms on large datasets.

- a) Explain how brute-force convex hull identifies boundary points.
- b) Analyze the time complexity.
- c) Discuss inefficiency for large datasets.

Q125. A document processing system scans large files using brute-force string matching to detect keywords.

- a) Explain how the algorithm works.
- b) Analyze the time complexity.
- c) Discuss inefficiency for large inputs.

Q126. An inventory system searches for item IDs in an unsorted dataset using Linear Search.

- a) Explain how Linear Search is applied.
- b) Analyze performance issues.
- c) Suggest improvements and justify.

Q127. A logistics company computes distances between warehouses to identify the closest ones using brute-force computation.

- a) Explain how all pairs are evaluated.
- b) Derive the time complexity.
- c) Discuss scalability concerns.

Q128. A system log analyzer scans large logs to detect patterns using brute-force string matching.

- a) Explain the working of the algorithm.
- b) Analyze its time complexity.
- c) Justify the need for optimized approaches.

Q129. A payroll system uses Binary Search on sorted employee IDs, but frequent insertions disturb order.

- a) Explain Binary Search operation.
- b) Analyze the impact of unsorted data.
- c) Discuss maintaining sorted data for efficiency.

Q130. A drone navigation system determines object boundaries using brute-force convex hull computation.

- a) Explain how convex hull is computed.
- b) Analyze the time complexity.
- c) Discuss why the approach is not scalable.

Q131. A smart city monitoring system collects air quality data from sensors in random order and needs sorting before analysis.

- a) Analyze how Selection Sort and Bubble Sort would perform on this dataset.
- b) Compare their efficiency in terms of comparisons and swaps.
- c) Recommend the most suitable sorting technique with justification.

Q132. A logistics platform computes distances between multiple distribution centers using brute-force to find the closest pair.

- a) Explain how the brute-force closest pair algorithm works.
- b) Derive the time complexity.
- c) Discuss scalability issues as the number of centers increases.

Q133. A university examination system stores answer scripts data unsorted due to continuous updates and uses Linear Search for retrieval.

- a) Explain how Linear Search works in this scenario.
- b) Analyze best, average, and worst-case performance.
- c) Justify whether sorting would improve efficiency.

Q134. A disaster response application maps affected areas using coordinate points and determines the boundary using convex hull techniques.

- a) Explain how brute-force convex hull operates.
- b) Analyze its computational complexity.
- c) Discuss limitations when handling large datasets.

Q135. A digital forensics system scans large text logs using brute-force string matching to detect suspicious patterns.

- a) Explain how the pattern matching algorithm works.
- b) Analyze its time complexity.
- c) Discuss inefficiency for large-scale data.

Q136. An e-commerce system searches for customer orders stored in an unsorted list using Linear Search.

- a) Explain how Linear Search operates in this system.
- b) Analyze performance degradation with increasing data size.
- c) Suggest improvements and justify.

Q137. A navigation system evaluates distances between multiple delivery locations to identify the closest pair using brute-force computation.

- a) Explain how all possible pairs are evaluated.

- b) Derive the time complexity.
- c) Discuss scalability challenges.

Q138. A software development tool searches for specific patterns in large codebases using brute-force string matching.

- a) Explain how the matching process works.
- b) Analyze its time complexity.
- c) Justify the need for optimized algorithms.

Q139. A banking system maintains sorted transaction IDs for quick lookup using Binary Search, but frequent updates disrupt the order.

- a) Explain how Binary Search works.
- b) Analyze the effect of unsorted data on performance.
- c) Discuss the importance of maintaining sorted order.

Q140. A robotics system processes coordinate points to determine object boundaries using brute-force convex hull computation.

- a) Explain how convex hull is determined.
- b) Analyze the time complexity.
- c) Discuss scalability concerns.

Q141. A financial analytics system collects fluctuating stock prices in random order and needs sorting before analysis.

- a) Compare Selection Sort and Bubble Sort for this dataset.
- b) Analyze their computational cost.
- c) Recommend the better algorithm with justification.

Q142. A telecom infrastructure system evaluates distances between communication towers to optimize connectivity using brute-force closest pair computation.

- a) Explain how the brute-force method works.
- b) Derive the time complexity.
- c) Discuss performance limitations.

Q143. A hospital database stores patient records unsorted and retrieves them using Linear Search during emergencies.

- a) Explain how Linear Search operates.
- b) Analyze its efficiency.
- c) Justify whether sorting is beneficial.

Q144. A geographic mapping system determines terrain boundaries using convex hull algorithms on large datasets.

- a) Explain how brute-force convex hull identifies boundary points.
- b) Analyze the time complexity.
- c) Discuss inefficiency for large datasets.

Q145. A document analysis system scans large files using brute-force string matching to detect keywords.

- a) Explain how the algorithm works.
- b) Analyze the time complexity.
- c) Discuss inefficiency for large inputs.

Q146. An inventory management system searches for product IDs in an unsorted dataset using Linear Search.

- a) Explain how Linear Search is applied.
- b) Analyze performance issues.
- c) Suggest improvements and justify.

Q147. A logistics company computes distances between delivery hubs to identify the closest ones using brute-force computation.

- a) Explain how all pairs are evaluated.
- b) Derive the time complexity.
- c) Discuss scalability concerns.

Q148. A system log analyzer scans large logs to detect patterns using brute-force string matching.

- a) Explain the working of the algorithm.
- b) Analyze its time complexity.
- c) Justify the need for optimized approaches.

Q149. A payroll system uses Binary Search on sorted employee IDs, but frequent insertions disturb order.

- a) Explain Binary Search operation.
- b) Analyze the impact of unsorted data.
- c) Discuss maintaining sorted data for efficiency.

Q150. A drone mapping system identifies terrain boundaries using brute-force convex hull computation.

- a) Explain how convex hull is computed.
- b) Analyze the time complexity.
- c) Discuss why the approach is not scalable.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity,	10	Clear, well-	Clear	Limited	Poorly

Justification & Presentation		justified explanation with strong reasoning	explanatio n with some justification	clarity and weak justification	explained with no justification
------------------------------------	--	--	---	--------------------------------------	------------------------------------